

Architecture of the Intelligent Autonomous Robot R2D2

Technical Documentation

Prepared by:

Islem Fakhfekh

Saif Eddine Ben Turkia

Asma Mhatli

Mohamed Amine Abderrazek

November 23, 2025

Contents

1	Introduction: Concept and Vision	3
2	Main Architecture Components	3
2.1	Context Broker NGSI v2 (FIWARE Orion) - The Backbone	3
2.1.1	Context Broker Roles:	3
2.2	FIROS Agent - The ROS $\leftrightarrow$ Context Broker Interface	4
2.3	ROS 2 - The Robot's Body	4
2.3.1	Audio Package (Voice Interaction)	4
2.3.2	Video Package (Vision and Streaming)	4
2.3.3	Navigation Package (SLAM, Nav2)	5
2.3.4	Perception Package (Multi-modal Sensors)	6
2.4	Intelligent Decision-Making with LLM and MCP	6
2.4.1	LLM Architecture:	6
2.4.2	Model Context Protocol (MCP):	7
2.4.3	Decision Flow:	7
2.4.4	N8N Alternative:	7
2.5	Digital Twin and Semantic Mapping	7
2.5.1	Digital Twin Components:	7
2.5.2	Digital Twin Usage:	8
2.6	Hybrid Network Architecture	8
2.6.1	Embedded Part (Robot - real-time):	8
2.6.2	External Part (Server - intelligence):	8
2.6.3	Communication:	8
3	Detailed Hardware Component Roles	9
3.1	Single-Board Computer (Raspberry Pi 4 or similar)	9
3.1.1	Responsibilities:	9
3.2	HC-SR04 Ultrasonic Sensors	9
3.2.1	Function:	9
3.3	USB Webcams with Integrated Microphone	9
3.3.1	Vision:	9
3.3.2	Audio:	10
3.4	L298N Motor Driver	10
3.4.1	Function:	10
4	Typical Interaction Flows	10
4.1	Scenario 1: Autonomous Navigation by Voice Command	10
4.2	Scenario 2: Object Detection and Recognition	11
4.3	Scenario 3: Reactive Safety with Ultrasound	11
5	Detailed ROS 2 Packages	11
5.1	Package <code>r2d2_audio</code>	11
5.2	Package <code>r2d2_vision</code>	11
5.3	Package <code>r2d2_navigation</code>	12

5.4	Package <code>r2d2_perception</code>	12
5.5	Package <code>r2d2_firos_bridge</code>	12
5.6	Package <code>r2d2_mcp_interface</code>	12
6	Synthesis: A Coherent and Intelligent Robotic System	12
7	References	13

1 Introduction: Concept and Vision

The R2D2 robot is an **autonomous and intelligent robotic system** capable of making independent decisions. Its name evokes its ability to discover its environment and interact naturally with humans through dialogue. At the heart of this architecture lies the **Context Broker NGSI v2**, which acts as the system's backbone - comparable to the spinal cord connecting the brain (external artificial intelligence) to the limbs (ROS actuators), sensory organs (sensors), and orchestrating all components for coherent and intelligent behavior.

The architecture relies on a **hybrid distributed approach** where:

- The **robot's body** (ROS 2 platform) handles motor and sensory functions in real-time
- The **intelligent brain** (external servers with LLM, vision AI, Digital Twin) processes complex decisions and contextual understanding
- The **nervous system** (FIWARE Context Broker) ensures coordination and shared memory between all components

This architecture enables R2D2 to function as a coherent system where each component collaborates via the Context Broker to create truly autonomous distributed intelligence.

2 Main Architecture Components

2.1 Context Broker NGSI v2 (FIWARE Orion) - The Backbone

The **Context Broker** is the nerve center of the R2D2 system. It doesn't simply coordinate data - it **coordinates the robot itself** as a unified intelligent system.

2.1.1 Context Broker Roles:

- **Central memory:** Maintains the context of detected objects, recognized persons, learned behaviors, and robot state
- **Digital Twin (DT):** Defines and maintains a digital representation of the robot's physical world, including object positions, environment map, and all system states
- **Intelligent coordination:** Orchestrates interactions between ROS 2 (body), AI systems (brain), and various services
- **Communication hub:** All important information passes through the broker as NGSI v2 entities (robot position, sensor status, detected objects, behavioral intentions, etc.)

The Digital Twin stored in the broker enables the robot to:

- Maintain a 3D representation of its environment with precise object positions

- Memorize interaction contexts (people encountered, conversations, preferences)
- Maintain coherence between what the robot perceives, understands, and plans

2.2 FIROS Agent - The ROS $\leftrightarrow$ Context Broker Interface

FIROS is a FIWARE agent that converts ROS inputs/outputs to the Context Broker in NGSI v2 format. Installed on the same server as the Context Broker, FIROS acts as a bidirectional translator:

- **ROS $\rightarrow$ Context Broker:** Transforms ROS topics (position /odom, sensors /ultrasonic, images /camera) into NGSI v2 entities in Orion
- **Context Broker $\rightarrow$ ROS:** Converts broker commands and updates (movement orders, navigation objectives) into ROS messages

FIROS's **publish/subscribe** mechanism enables real-time interaction with the robot: any entity modification in the broker can instantly trigger an action on the robot, and any sensor data immediately rises to the global context.

2.3 ROS 2 - The Robot's Body

ROS 2 constitutes the robot's peripheral nervous system, managing motor and sensory functions in real-time. The ROS 2 architecture is divided into **four specialized package families**:

2.3.1 Audio Package (Voice Interaction)

This package handles bidirectional voice communication with the user via an optimized pipeline:

- **Direct Speech-to-LLM:** Rather than Speech-to-Text then Text-to-LLM, we favor a direct connection from audio to LLM for better performance and superior contextual understanding
- **LLM-to-Speech:** Natural voice synthesis generated directly from the LLM
- **Open-source solutions:** Use of Vosk, DeepSpeech, or Whisper for STT, and eSpeak/Piper for TTS
- **Integrated microphone:** The webcam's microphone captures voice commands, published to the Context Broker via ROS

2.3.2 Video Package (Vision and Streaming)

Vision is managed in two stages: local capture and remote analysis.

Capture (Local ROS):

- `usb_cam` or `libcamera_ros` nodes capture images from two USB webcams
- Publication on `/camera/image_raw` topics via `image_transport`
- Light preprocessing (OpenCV) for compression and filtering

Analysis (External Server):

- **Both options will be evaluated:** image analysis on ROS (embedded) vs external AI server
- Since the robot's brain is remote, **might as well centralize intelligent systems** on the server (vision AI, LLM, Digital Twin)
- External AI service (TensorFlow/PyTorch) for object detection (YOLO), facial recognition, and depth estimation
- Results (detected objects, 3D positions, identifications) sent to the Context Broker
- Real-time visualization possible via RViz, Foxglove, or web interface

Vision serves two major objectives:

1. **Recognition:** Identify objects and people
2. **Spatial localization:** Accurately calculate the distance and orientation of objects for navigation and manipulation

2.3.3 Navigation Package (SLAM, Nav2)

This package implements the robot's autonomous mobility.

SLAM (Mapping):

- SLAM node (RTAB-Map or SLAM Toolbox) builds a 2D/3D map in real-time
- Data fusion: webcam images, ultrasound, odometry
- Map published in the Context Broker as "Map" entity of the Digital Twin

Nav2 (Autonomous Navigation):

- ROS 2 navigation stack with modular servers: path planning, trajectory tracking, obstacle avoidance
- Orchestration via behavior trees
- Inputs: map, robot position, sensor data
- Outputs: velocity commands (`geometry_msgs/Twist`) to motors

Interaction with Digital Twin:

- SLAM map feeds the Digital Twin in the Context Broker
- Object positions detected by vision are integrated into the DT
- Nav2 consumes the global context (via FIROS) for intelligent navigation considering the semantic environment (e.g., avoiding a recognized person)

2.3.4 Perception Package (Multi-modal Sensors)

Ultrasonic Sensors (HC-SR04):

- Connected directly to the Raspberry Pi (simplified mounting)
- Function: **reactive safety** for emergency stop or automatic obstacle avoidance
- No AI call required for this safety function
- Measurements published on `/ultrasonic` topic and sent to Context Broker for global system information

Webcams (Vision):

- Provide raw images for the video pipeline described above
- Enable stereo depth estimation if two cameras are properly calibrated

Perception data (distances, images, depth) are consolidated in the Context Broker to feed the Digital Twin and decision systems.

2.4 Intelligent Decision-Making with LLM and MCP

The robot integrates a **cognitive decision layer** based on a Large Language Model.

2.4.1 LLM Architecture:

- **Local LLM** hosted on the server (robot extension/brain, not really "cloud")
- Possible models: Claude (via MCP), GPT, LLaMA, Mistral
- The LLM acts as cognitive brain: natural language understanding, mission planning, contextual decision-making

2.4.2 Model Context Protocol (MCP):

- Standard protocol to connect the LLM to the robot
- The MCP Server exposes **tools** (Python functions) to the LLM:
 - `get_camera_frame()`: Capture an image
 - `move_robot(direction, speed)`: Command a movement
 - `query_context(entity)`: Query the Digital Twin
 - `detect_objects()`: Trigger visual analysis
- The LLM can "see, reason, and act" by calling these tools according to context

2.4.3 Decision Flow:

1. User speaks to the robot (audio → Speech-to-LLM)
2. The LLM understands the intention and consults the Context Broker (Digital Twin)
3. The LLM plans an action sequence and calls MCP tools
4. Tools interact with ROS (via FIROS/Context Broker) to execute physical actions
5. The LLM generates a vocal response (LLM-to-Speech)

2.4.4 N8N Alternative:

- Automation platform to orchestrate complex workflows
- Can complement or replace MCP for scenarios requiring integration with external APIs, databases, etc.

2.5 Digital Twin and Semantic Mapping

The **Digital Twin** is the complete digital representation of the environment and robot state, hosted in the Context Broker.

2.5.1 Digital Twin Components:

- **Spatial map**: 2D/3D occupancy grid generated by SLAM
- **Contextualized objects**: Position, type, orientation of each detected object (e.g., "chair at 2m in front of robot, oriented at 45°")
- **Social context**: Identified persons, interaction history
- **Robot state**: Position, velocity, battery charge, sensor status
- **Behavioral memory**: Learned preferences, frequent routes, forbidden zones

2.5.2 Digital Twin Usage:

- The LLM queries the DT to answer complex questions: "Is there a clear path to the kitchen?" or "Where did I see Marie last time?"
- Nav2 uses the enriched DT (objects + semantics) for intelligent navigation
- AI systems continuously update the DT with their discoveries

2.6 Hybrid Network Architecture

The system operates in **hybrid distributed architecture**:

2.6.1 Embedded Part (Robot - real-time):

- ROS 2 platform on single-board computer (Raspberry Pi 4 or equivalent)
- Handles critical real-time functions: sensor reading, motor control, SLAM, obstacle avoidance
- Wi-Fi or Ethernet connection to local network

2.6.2 External Part (Server - intelligence):

- Physical or cloud server hosting:
 - FIWARE Orion Context Broker (NGSI v2)
 - FIROS Agent
 - LLM with MCP server
 - AI services (vision, recognition)
 - Digital Twin
- Containerization recommended (Docker) for easy deployment

2.6.3 Communication:

- ROS 2 uses its DDS middleware for decentralized network communication
- FIROS ensures real-time gateway between ROS and Context Broker
- Network loss-tolerant architecture thanks to NGSI pub/sub

Important note: This isn't really "cloud" in the public Internet sense, but rather **the robot's cerebral extension** - a local or remote server that constitutes the system's intelligent brain.

3 Detailed Hardware Component Roles

3.1 Single-Board Computer (Raspberry Pi 4 or similar)

Role: Main embedded computer running ROS 2

3.1.1 Responsibilities:

- Hosting all critical ROS 2 nodes (Audio, Video, Navigation, Perception packages)
- Executing SLAM and Nav2 for local autonomous navigation
- Initial image processing (capture, compression)
- Reading ultrasonic sensors
- Generating PWM commands for motors
- Network communication (Wi-Fi) with server
- Temporary storage (SLAM map, image buffers)

Note: The architecture is board-independent - any ROS 2-compatible system (Jetson Nano, Intel NUC, etc.) can replace the Raspberry Pi.

3.2 HC-SR04 Ultrasonic Sensors

Role: Reactive proximity safety

Connection: Directly to Raspberry Pi GPIO (simplification)

3.2.1 Function:

- Measure distance to nearby obstacles (0.02m to 4m)
- Trigger emergency stop or automatic obstacle avoidance **without AI call**
- Publish measurements on ROS topics `/ultrasonic_front`, `/ultrasonic_rear`
- Upload to Context Broker for global system information

3.3 USB Webcams with Integrated Microphone

Dual role: Vision and audio

3.3.1 Vision:

- Video capture for object detection, recognition, visual mapping
- Images published on `/camera/image_raw`
- Stereovision capability for depth estimation

3.3.2 Audio:

- Capture user voice commands
- Feed Speech-to-LLM pipeline
- Continuous listening with activation detection (wake word possible)

3.4 L298N Motor Driver

Role: Power interface for motors

Connection: Receives PWM signals from Raspberry Pi

3.4.1 Function:

- Control two DC motors (wheels)
- Control speed and rotation direction
- Convert ROS commands (`geometry_msgs/Twist`) to motor voltages

Command flow: Nav2 → ROS topic `/cmd_vel` → Motor node (ROS) → GPIO PWM (Pi) → L298N → Motors

4 Typical Interaction Flows

4.1 Scenario 1: Autonomous Navigation by Voice Command

1. **User:** "R2D2, go to the kitchen"
2. **Audio** → **LLM:** Microphone captures audio → Direct Speech-to-LLM
3. **LLM** → **Context Broker:** The LLM queries the Digital Twin: "Where is the kitchen?" and "What is the robot's current position?"
4. **Planning:** The LLM calls MCP tool `navigate_to(location="kitchen")`
5. **MCP** → **FIROS** → **ROS:** The command is translated to NGSI entity then to Nav2 goal
6. **Nav2 executes:** Path planning, obstacle avoidance (ultrasound + map)
7. **Feedback:** Position continuously updated in Context Broker
8. **Confirmation:** Upon arrival, LLM generates "I've arrived at the kitchen" → LLM-to-Speech

4.2 Scenario 2: Object Detection and Recognition

1. **Autonomous exploration:** R2D2 navigates in discovery mode
2. **Capture:** Webcam captures an image → published on `/camera/image_raw`
3. **External AI analysis:** Image sent to AI service on server
4. **Detection:** Service detects "person" with 3D position (2.5m at 30° right)
5. **Context Broker:** New object added to Digital Twin: entity "Person_1"
6. **LLM notified:** The LLM can decide to interact: "Hello, can I help you?"
7. **Adaptive navigation:** Nav2 automatically avoids this person thanks to updated DT

4.3 Scenario 3: Reactive Safety with Ultrasound

1. **R2D2 is moving** (Nav2 command in progress)
2. **Sudden obstacle:** An object appears at 15cm (detected by front ultrasound)
3. **Immediate reaction** (without AI): Ultrasound node publishes alert
4. **Automatic stop:** Motor controller (ROS) stops motors
5. **Information:** Event published to Context Broker
6. **LLM informed:** LLM can say "Warning, obstacle detected" and decide to bypass

5 Detailed ROS 2 Packages

5.1 Package `r2d2_audio`

- Node `audio_input`: Microphone capture, speech detection
- Node `speech_to_llm`: Direct audio transmission to LLM via API
- Node `llm_to_speech`: Voice synthesis from LLM responses
- Topics: `/audio_raw`, `/speech_recognized`, `/speech_output`

5.2 Package `r2d2_vision`

- Node `camera_capture`: USB image acquisition (`usb_cam`)
- Node `image_preprocessor`: Filtering and compression (OpenCV)
- Node `vision_ai_client`: Send images to AI server, receive detections
- Topics: `/camera/image_raw`, `/camera/image_processed`, `/vision/detections`

5.3 Package r2d2_navigation

- **Node** `slam_node`: SLAM Toolbox or RTAB-Map
- **Nav2 Stack**: `map_server`, `amcl`, `planner_server`, `controller_server`, `bt_navigator`
- **Topics**: `/map`, `/odom`, `/scan` (if lidar), `/cmd_vel`, `/goal_pose`

5.4 Package r2d2_perception

- **Node** `ultrasonic_node`: HC-SR04 reading, distance publication
- **Node** `safety_monitor`: Ultrasound monitoring for emergency stop
- **Topics**: `/ultrasonic_front`, `/ultrasonic_rear`, `/safety_alert`

5.5 Package r2d2_firos_bridge

- **Node** `firos_adapter`: Interface with FIROS agent
- **Configuration**: Mapping ROS topics ↔ NGSI entities (Robot, Sensors, Map, Objects)
- **Special topics**: `/firos/robot_state`, `/firos/commands`

5.6 Package r2d2_mcp_interface

- **Node** `mcp_server`: MCP server exposing Python tools to LLM
- **Available tools**:
 - `get_camera_frame()`, `move_robot()`, `get_position()`
 - `query_context()`, `detect_objects()`, `navigate_to()`
 - `say()`, `get_map()`
- **Communication**: WebSocket or HTTP to external LLM

6 Synthesis: A Coherent and Intelligent Robotic System

The R2D2 robot architecture rests on three interconnected pillars:

1. **The Body (ROS 2)**: Handles motor and sensory functions in real-time with maximum reactivity
2. **The Brain (LLM + External AI)**: Provides intelligence, contextual understanding, and high-level planning

3. **The Nervous System (Context Broker + Digital Twin):** Ensures coordination, shared memory, and global coherence

This distributed architecture enables R2D2 to function as a **truly autonomous system**, capable of:

- Perceiving its environment (vision + ultrasound)
- Understanding human intentions (Speech-to-LLM)
- Reasoning about its actions (LLM + Digital Twin)
- Acting in a coordinated manner (Nav2 + MCP)
- Learning and memorizing (persistent Context Broker)

All components collaborate via the Context Broker to create emergent intelligence exceeding the sum of its parts - a robot that doesn't just execute orders, but discovers, understands, and decides autonomously.

7 References

- [1] Robot Operating System 2 (ROS 2) Architecture | Medium
<https://medium.com/software-architecture-foundations/robot-operating-system-2-ro>
- [2] mycroft_ros - ROS Wiki
http://wiki.ros.org/mycroft_ros
- [3] Nav2 Documentation
<https://docs.nav2.org/>
- [4] GitHub - iml130/firos
<https://github.com/iml130/firos>
- [5] MCP for Robotics - Phospho Documentation
<https://docs.phospho.ai/examples/mcp-for-robotics>
- [6] LLM Agents Guide - n8n Blog
<https://blog.n8n.io/llm-agents/>
- [7] FogROS2 - Berkeley Autolab
https://autolab.berkeley.edu/assets/publications/media/2023_ICRA_FogROS2_Camera_Ready.pdf